

THỰC HÀNH THIẾT KẾ HỆ THỐNG SOC - LỚP **CE433.Q21.1**

BÀI THỰC HÀNH 6: GIAO TIẾP I/O = LCD

Giảng viên hướng dẫn	Trương Văn Cường		ĐIỂM
Sinh viên thực hiện	Vũ Đại Dương	23520359	

Bài 1: Thực hiện bộ đếm từ 00 – 99 trên LCD

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opcode Name
☒		☐ clk_0 ☐ clk_in ☐ clk_in_reset ☐ clk ☐ clk_reset	Clock Source Clock Input Reset Input Clock Output Reset Output	clk reset Double-click to export Double-click to export	clk_0				
☒		☐ onchip_memory2_0 ☐ clk1 ☐ s1 ☐ reset1	On-Chip Memory (RAM or ROM) Clock Input Avalon Memory Mapped Slave Reset Input	Double-click to export Double-click to export Double-click to export	clk_0 [clk1]	# 0x0000_8000	0x0000_ffff		
☒		☐ nios2_qsysa_0 ☐ clk ☐ reset_n ☐ data_master ☐ instruction_master ☐ jtag_debug_module_re ☐ jtag_debug_module ☐ custom_instruction_m	Nios II Processor Clock Input Reset Input Avalon Memory Mapped Master Avalon Memory Mapped Master Reset Output Avalon Memory Mapped Slave Custom Instruction Master	Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk] [clk] [clk] [clk]		IRQ 0	IRQ 31	
☒		☐ jtag_uart_0 ☐ clk ☐ reset ☐ avalon_jtag_slave	JTAG UART Clock Input Reset Input Avalon Memory Mapped Slave	Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk]	# 0x0001_1060	0x0001_1067		
☒		☐ lcd_on ☐ clk ☐ reset ☐ s1 ☐ external_connection	PIO (Parallel IO) Clock Input Reset Input Avalon Memory Mapped Slave Conduit	Double-click to export Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk]	# 0x0001_1050	0x0001_105f		
☒		☐ lcd_blon ☐ clk ☐ reset ☐ s1 ☐ external_connection	PIO (Parallel IO) Clock Input Reset Input Avalon Memory Mapped Slave Conduit	Double-click to export Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk]	# 0x0001_1040	0x0001_104f		
☒		☐ lcd_en ☐ clk ☐ reset ☐ s1 ☐ external_connection	PIO (Parallel IO) Clock Input Reset Input Avalon Memory Mapped Slave Conduit	Double-click to export Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk]	# 0x0001_1030	0x0001_103f		
☒		☐ lcd_rw ☐ clk ☐ reset ☐ s1 ☐ external_connection	PIO (Parallel IO) Clock Input Reset Input Avalon Memory Mapped Slave Conduit	Double-click to export Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk]	# 0x0001_1020	0x0001_102f		
☒		☐ lcd_rs ☐ clk ☐ reset ☐ s1 ☐ external_connection	PIO (Parallel IO) Clock Input Reset Input Avalon Memory Mapped Slave Conduit	Double-click to export Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk]	# 0x0001_1010	0x0001_101f		
☒		☐ lcd_d ☐ clk ☐ reset ☐ s1 ☐ external_connection	PIO (Parallel IO) Clock Input Reset Input Avalon Memory Mapped Slave Conduit	Double-click to export Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk]	# 0x0001_1000	0x0001_100f		

Hình 1. Thiết kế HLS

Sử dụng các PIO output để truyền tín hiệu điều khiển đến các chân của LCD HD44780.

Thiết kế top-level kết nối các chân vật lý của board với HLS.

```

module lab5_part1 (
    input CLOCK_50,
    input [0:0] KEY,
    output LCD_RW,
    output LCD_EN,
    output LCD_RS,
    output LCD_ON,
    output LCD_BLON,

```

```

        output [7:0] LCD_DATA
    );

    system u0 (
        .clk_clk (CLOCK_50),
        .reset_reset_n (KEY[0]),
        .lcd_on_export (LCD_ON),
        .lcd_blon_export (LCD_BLON),
        .lcd_en_export (LCD_EN),
        .lcd_rw_export (LCD_RW),
        .lcd_rs_export (LCD_RS),
        .lcd_d_export (LCD_DATA)
    );
endmodule

```

Sử dụng code C để điều khiển hệ thống.

```

#include "sys/alt_stdio.h"
#include <stdio.h>
#include "system.h"
#include "altera_avalon_pio_regs.h"

void delay(int a) {
    volatile int i = 0;
    while (i < a * 10000) {
        i++;
    }
}

void command(char data) {
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_RS_BASE, 0x00);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_RW_BASE, 0x00);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_D_BASE, data & 0xFF);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_EN_BASE, 0x01);
    delay(20);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_EN_BASE, 0x00);
    delay(20);
}

void lcd_data(char data) {
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_RS_BASE, 0x01);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_RW_BASE, 0x00);
}

```

```

IOWR_ALTERA_AVALON_PIO_DATA(LCD_D_BASE, data & 0xFF);
IOWR_ALTERA_AVALON_PIO_DATA(LCD_EN_BASE, 0x01);
delay(20);
IOWR_ALTERA_AVALON_PIO_DATA(LCD_EN_BASE, 0x00);
delay(20);
}

void lcd_string(char *str) {
    char i = 0;
    while (str[i] != 0) {
        lcd_data(str[i]);
        i++;
    }
}

void lcd_init() {
    command(0x38);
    delay(100);
    command(0x0c);
    delay(100);
    command(0x06);
    delay(100);
    command(0x01);
}

int main() {
    int i, j;
    char data[10] = {'0','1','2','3','4','5','6','7','8','9'};

    lcd_init();

    IOWR_ALTERA_AVALON_PIO_DATA(LCD_ON_BASE, 0x01);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_BLON_BASE, 0x01);

    while (1) {
        for (i = 0; i < 100; i++) {
            command(0x02);
            lcd_data(data[i / 10]);
            lcd_data(data[i % 10]);
            delay(100);
        }
    }
}

```

```
return 0;  
}
```

Sử dụng void delay để tạo độ trễ (a) ms để dễ theo dõi và các lệnh có thời gian hoàn tất xử lí.

Hàm void command gửi lệnh điều khiển đến LCD, với:

- RS = 0 → Báo cho LCD là lệnh
- RW = 0 → Chọn chế độ Write
- Đưa dữ liệu ra chân LCD_D
- Chuyển EN 1 → 0 để chốt dữ liệu vào LCD.

Void lcd_data để gửi dữ liệu đến LCD

- RS = 1 → Báo cho LCD là kí tự
- Các lệnh còn lại như command

Void lcd_string để in một chuỗi kí tự dài.

Void lcd_init để khởi tạo màn hình LCD, thực hiện gửi các mã lệnh HEX chuẩn theo datasheet của LCD HD44780.

- 0x38: Thiết lập chế độ giao tiếp 8-bit, hiển thị 2 dòng, font chữ 5x8.
- 0x0c: Bật màn hình, tắt cursor.
- 0x06: Chế độ tự động tăng địa chỉ con trỏ sang phải sau khi in mỗi ký tự.
- 0x01: Xóa trắng màn hình.

Trong hàm main, thực hiện khởi tạo mảng giá trị từ 0 đến 9, khởi tạo LCD với lcd_init, thực hiện bật nguồn LCD (LCD_ON lên 1) và bật đèn nền (LCD_BLON lên 1). Thực hiện đếm từ 0 đến 99 với các bước: xóa hiển thị LCD với command(0x02) và hiển thị số chục, số đơn vị qua xử lí lcd_data(data[i/10]), lcd_data(data[i%10]) và tạo độ trễ để theo dõi giá trị.

Bài 2: 1 Button và LCD đếm số lần nhấn nút từ 00 - 99

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opcode Name
☒		clk_in_reset	Reset Input	reset	clk_0				
☒		clk	Clock Output	Double-click to export	clk_0				
☒		clk_reset	Reset Output	Double-click to export	clk_0				
☒		onchip_memory2_0	On-Chip Memory (RAM or ROM)	Double-click to export	clk_0				
☒		clk1	Clock Input	Double-click to export	clk_0				
☒		s1	Avalon Memory Mapped Slave	Double-click to export	clk_0				
☒		reset1	Reset Input	Double-click to export	clk_0				
☒		nios2_qsys_0	Nios II Processor	Double-click to export	clk_0				
☒		clk	Clock Input	Double-click to export	clk_0				
☒		reset_n	Reset Input	Double-click to export	clk_0				
☒		data_master	Avalon Memory Mapped Master	Double-click to export	clk_0				
☒		instruction_master	Avalon Memory Mapped Master	Double-click to export	clk_0				
☒		jtag_debug_module_re	Reset Output	Double-click to export	clk_0				
☒		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export	clk_0				
☒		custom_instruction_m	Custom Instruction Master	Double-click to export	clk_0				
☒		jtag_uart_0	JTAG UART	Double-click to export	clk_0				
☒		clk	Clock Input	Double-click to export	clk_0				
☒		reset	Reset Input	Double-click to export	clk_0				
☒		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	clk_0				
☒		LCD_ON	PIO (Parallel IO)	Double-click to export	clk_0				
☒		clk	Clock Input	Double-click to export	clk_0				
☒		reset	Reset Input	Double-click to export	clk_0				
☒		s1	Avalon Memory Mapped Slave	Double-click to export	clk_0				
☒		external_connection	Conduit	lcd_on	clk_0				
☒		LCD_BLON	PIO (Parallel IO)	Double-click to export	clk_0				
☒		clk	Clock Input	Double-click to export	clk_0				
☒		reset	Reset Input	Double-click to export	clk_0				
☒		s1	Avalon Memory Mapped Slave	Double-click to export	clk_0				
☒		external_connection	Conduit	lcd_blon	clk_0				
☒		LCD_EN	PIO (Parallel IO)	Double-click to export	clk_0				
☒		clk	Clock Input	Double-click to export	clk_0				
☒		reset	Reset Input	Double-click to export	clk_0				
☒		s1	Avalon Memory Mapped Slave	Double-click to export	clk_0				
☒		external_connection	Conduit	lcd_en	clk_0				
☒		LCD_RW	PIO (Parallel IO)	Double-click to export	clk_0				
☒		clk	Clock Input	Double-click to export	clk_0				
☒		reset	Reset Input	Double-click to export	clk_0				
☒		s1	Avalon Memory Mapped Slave	Double-click to export	clk_0				
☒		external_connection	Conduit	lcd_rw	clk_0				
☒		LCD_RS	PIO (Parallel IO)	Double-click to export	clk_0				
☒		clk	Clock Input	Double-click to export	clk_0				
☒		reset	Reset Input	Double-click to export	clk_0				
☒		s1	Avalon Memory Mapped Slave	Double-click to export	clk_0				
☒		external_connection	Conduit	lcd_rs	clk_0				
☒		LCD_D	PIO (Parallel IO)	Double-click to export	clk_0				
☒		clk	Clock Input	Double-click to export	clk_0				
☒		reset	Reset Input	Double-click to export	clk_0				
☒		s1	Avalon Memory Mapped Slave	Double-click to export	clk_0				
☒		external_connection	Conduit	lcd_d	clk_0				
☒		key1	PIO (Parallel IO)	Double-click to export	clk_0				
☒		clk	Clock Input	Double-click to export	clk_0				
☒		reset	Reset Input	Double-click to export	clk_0				
☒		s1	Avalon Memory Mapped Slave	Double-click to export	clk_0				
☒		external_connection	Conduit	key1	clk_0				

Hình 2. Thiết kế HLS

Thêm PIO Input tên key1 để nhận dữ liệu truyền vào từ KEY1

Thiết kế top-level kết nối các chân vật lý của board với HLS.

```

module lab5_part2 (
    input CLOCK_50,
    input [1:0] KEY,
    output LCD_RW,
    output LCD_EN,
    output LCD_RS,
    output LCD_ON,
    output LCD_BLON,
    output [7:0] LCD_DATA
);

system u0 (
    .clk_clk (CLOCK_50),
    .reset_reset_n (KEY[0]),
    .lcd_on_export (LCD_ON),
    .lcd_blon_export (LCD_BLON),
    .lcd_en_export (LCD_EN),
    .lcd_rw_export (LCD_RW),
    .lcd_rs_export (LCD_RS),

```

```
        .lcd_d_export (LCD_DATA),  
        .key1_export (KEY[1])  
    );  
endmodule
```

Sử dụng code C để điều khiển hệ thống

```
/* source.c */  
  
#include "sys/alt_stdio.h"  
#include <stdio.h>  
#include "system.h"  
#include "altera_avalon_pio_regs.h"  
  
void delay(int a) {  
    volatile int i = 0;  
    while (i < a * 10000) {  
        i++;  
    }  
}  
  
void command(char data) {  
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_RS_BASE, 0x00);  
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_RW_BASE, 0x00);  
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_D_BASE, data & 0xFF);  
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_EN_BASE, 0x01);  
    delay(20);  
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_EN_BASE, 0x00);  
    delay(20);  
}  
  
void lcd_data(char data) {  
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_RS_BASE, 0x01);  
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_RW_BASE, 0x00);  
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_D_BASE, data & 0xFF);  
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_EN_BASE, 0x01);  
    delay(20);  
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_EN_BASE, 0x00);  
    delay(20);  
}  
  
void lcd_string(char *str) {
```

```

char i = 0;
while (str[i] != 0) {
    lcd_data(str[i]);
    i++;
}
}

void lcd_init() {
    command(0x38);
    delay(100);
    command(0x0c);
    delay(100);
    command(0x06);
    delay(100);
    command(0x01);
}

int main() {
    int i = 0;
    char data[10] = {'0','1','2','3','4','5','6','7','8','9'};

    lcd_init();
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_ON_BASE, 0x01);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_BLON_BASE, 0x01);

    // Khoi tao hien thi 00
    command(0x02);
    lcd_data(data[0]);
    lcd_data(data[0]);

    while(1) {
        // Kiem tra nut nhan
        if (IORD_ALTERA_AVALON_PIO_DATA(KEY1_BASE) == 0) {
            delay(5); // Chong rung phim

            if (IORD_ALTERA_AVALON_PIO_DATA(KEY1_BASE) == 0) {
                i++;
                if (i >= 100) {
                    i = 0;
                }

                command(0x02);
                lcd_data(data[i / 10]);
            }
        }
    }
}

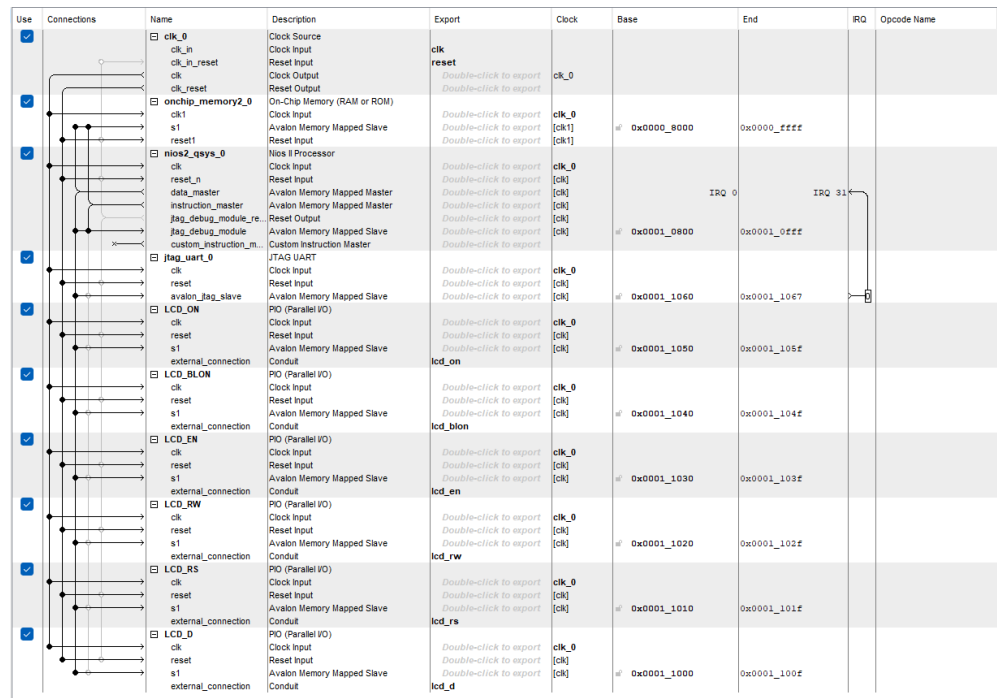
```

```
lcd_data(data[i % 10]);

// Cho nha phim
while(IORD_ALTERA_AVALON_PIO_DATA(KEY1_BASE) == 0);
}
}
}
return 0;
}
```

Trong hàm main, thực hiện khởi tạo như bài 1 và thêm phần hiển thị 00 lên màn hình. Hàm while thay vì sử dụng vòng for để thực hiện tăng giá trị như bài trước, thì bài này thực hiện tăng giá trị i khi KEY1 Active Low.

Bài 3: Thực hiện đồng hồ Giờ-Phút-Giây: hh-mm-ss; Ngày-Tháng-Năm (dd-mm-yyyy) trên LCD.



Hình 3. Thiết kế HLS

Hệ thống HLS được thiết kế như bài 1, với các chân PIO output để truyền dữ liệu đến LCD.

```
module lab5_part1 (
    input CLOCK_50,
    input [0:0] KEY,
```



```

        output LCD_RW,
        output LCD_EN,
        output LCD_RS,
        output LCD_ON,
        output LCD_BLON,
        output [7:0] LCD_DATA
    );

    system u0 (
        .clk_clk (CLOCK_50),
        .reset_reset_n (KEY[0]),
        .lcd_on_export (LCD_ON),
        .lcd_blon_export (LCD_BLON),
        .lcd_en_export (LCD_EN),
        .lcd_rw_export (LCD_RW),
        .lcd_rs_export (LCD_RS),
        .lcd_d_export (LCD_DATA)
    );
endmodule

```

Thiết kế top-level kết nối các chân vật lý của board với HLS.

Sử dụng code C để điều khiển hệ thống.

```

#include "sys/alt_stdio.h"
#include <stdio.h>
#include "system.h"
#include "altera_avalon_pio_regs.h"

void delay(int a) {
    volatile int i = 0;
    while (i < a * 10000) {
        i++;
    }
}

void command(char data) {
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_RS_BASE, 0x00);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_RW_BASE, 0x00);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_D_BASE, data & 0xFF);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_EN_BASE, 0x01);
    delay(20);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_EN_BASE, 0x00);
    delay(20);
}

```

```

}

void lcd_data(char data) {
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_RS_BASE, 0x01);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_RW_BASE, 0x00);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_D_BASE, data & 0xFF);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_EN_BASE, 0x01);
    delay(20);
    IOWR_ALTERA_AVALON_PIO_DATA(LCD_EN_BASE, 0x00);
    delay(20);
}

void lcd_string(char *str) {
    char i = 0;
    while (str[i] != 0) {
        lcd_data(str[i]);
        i++;
    }
}

void lcd_init() {
    command(0x38); delay(100);
    command(0x0c); delay(100);
    command(0x06); delay(100);
    command(0x01); delay(100);
}

// Ham tinh so ngay toi da trong thang (bao gom ca nam nhuan)
int get_max_day(int month, int year) {
    if (month == 4 || month == 6 || month == 9 || month == 11) return 30;
    if (month == 2) {
        if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) return 29;
        return 28;
    }
    return 31;
}

int main() {
    // Gia tri khoi tao ban dau
    int sec = 0, min = 0, hour = 0;
    int day = 7, month = 6, year = 2026;

    char data[10] = {'0','1','2','3','4','5','6','7','8','9'};

```

```

lcd_init();
IOWR_ALTERA_AVALON_PIO_DATA(LCD_ON_BASE, 0x01);
IOWR_ALTERA_AVALON_PIO_DATA(LCD_BLON_BASE, 0x01);

while (1) {
    // ---- Hang 1: Time: hh-mm-ss ----
    command(0x80); // Di chuyen con tro ve dau dong 1
    lcd_string("Time: ");
    lcd_data(data[hour / 10]);
    lcd_data(data[hour % 10]);
    lcd_data('-');
    lcd_data(data[min / 10]);
    lcd_data(data[min % 10]);
    lcd_data('-');
    lcd_data(data[sec / 10]);
    lcd_data(data[sec % 10]);

    // ---- Hang 2: Date: dd-mm-yyyy ----
    command(0xC0); // Di chuyen con tro ve dau dong 2
    lcd_string("Date: ");
    lcd_data(data[day / 10]);
    lcd_data(data[day % 10]);
    lcd_data('-');
    lcd_data(data[month / 10]);
    lcd_data(data[month % 10]);
    lcd_data('-');
    lcd_data(data[year / 1000]);
    lcd_data(data[(year / 100) % 10]);
    lcd_data(data[(year / 10) % 10]);
    lcd_data(data[year % 10]);

    // ---- Thoi gian tre de tao 1 giay ----
    delay(10);

    // ---- Logic tang thoi gian ----
    sec++;
    if (sec >= 60) {
        sec = 0;
        min++;
        if (min >= 60) {
            min = 0;
            hour++;
        }
    }
}

```

```

        if (hour >= 24) {
            hour = 0;
            day++;
            if (day > get_max_day(month, year)) {
                day = 1;
                month++;
                if (month > 12) {
                    month = 1;
                    year++;
                }
            }
        }
    }
}
return 0;
}

```

Khởi tạo giá trị với hh-mm-ss = 00-00-00, dd-mm-yyyy = 07-06-2026

Cấu hình phần cứng với hàm lcd_init, cấp nguồn (LCD_ON = 1) và bật đèn nền cho LCD (LCD_BLON = 1)

Trong vòng lặp while, thực hiện hiển thị ra lcd với:

- Dòng 1: Time: hh-mm-ss
- Dòng 2: Date: dd-mm-yyyy

Và thực hiện logic xử lý thời gian.

Video demo: [Video](#)